

Resilienzmuster im Linux Kernel

Muhamedbaqir Al-Rumeil

Duale Hochschule Baden-Württemberg Heidenheim
Marienstraße 20 89518 Heidenheim



Abstrakt

Resilienzmuster kommen meistens in verteilten Systemen zum Einsatz, mit dem Ziel die Verfügbarkeit eines Systems zu erhöhen. Eher unüblich sind diese Strategien im Gebiet der Betriebssysteme, geschuldet der Verschiedenheit der beiden Domänen.

Diese Studienarbeit befasst sich mit der Fragestellung, ob ein Transfer zwischen den beiden Bereichen stattfand, am Beispiel der im Linux Kernel implementierten Resilienzmuster. Wir kommen zu dem Ergebnis, dass der Linux Kernel diverse Resilienzstrategien implementiert, welche signifikant durch die quelloffene Arbeitsweise geprägt wurden.

Hintergründe

Resilienzmuster helfen Softwaresystemen, auch unter Fehlern, Lastspitzen oder Teilausfällen stabil und verfügbar zu bleiben. Typische Ansätze sind etwa Circuit Breaker (Vermeidung von Kettenreaktionen bei Dienstausschfällen), Retries mit Backoff (erneutes Ausführen fehlgeschlagener Aufrufe), Bulkheads (Abschirmung von Komponenten) oder Fallbacks (alternative Antworten im Fehlerfall). Durch den gezielten Einsatz solcher Muster können Systeme robuster, selbstheilend und nutzerfreundlicher gestaltet werden.

Ziele

Im Kontrast zu verteilten Systemen, fehlt einem monolithischen System wie dem Linux Kernel die Möglichkeit Funktionalität extern auszulagern, oder hohe Lasten umzulenken. Linux Entwickler haben dennoch die Option, kernelinterne Prozesse durch Resilienzstrategien abzusichern.

Der Linux Kernel dient als treffendes Beispiel für diese Untersuchung, da Quelltext, Diskussionen und Dokumentation transparent einsehbar sind. Desweiteren ist Linux besonders in Domänen erfolgreich, wo Resilienz am höchsten gefordert ist, beispielsweise in Cloud oder eingebetteten Systemen.

Unsere Strategie für die Analyse des Linux Kernels ist folgende:

- 1 Analyse der Systemstruktur anhand der Kernel Dokumentation und Quelltexte
- 2 Interaktionen der Systemkomponenten mit Resilienzmustern abgleichen
- 3 Experimentelle Demonstration diverser Fehlerszenarien und die Reaktion des Kernels

Ergebnisse

Bulkhead - Dekomposition

Der Linux Kernel implementiert in diversen Formen eine systematische Dekomposition, um einzuschränken wie Kernel Submodule einander beeinflussen können.

- Kapselung: Durch den Gebrauch von Header-Files wird bereits zur Kompilierzeit eingeschränkt, welche Schnittstellen einem Modul bereitstehen. Die entsprechenden Header Files sind entsprechend der diversen Subsysteme organisiert, sodass Kernel Programme zur Laufzeit einen drastisch reduzierten Handlungsraum haben.
- Trennung der Gerätetreiber: Gerätetreiber stellen den größten Teil des Quelltexts dar und bergen daher ein großes Risiko für Laufzeitfehler. Gerätetreiber des Linux Kernels werden strikt voneinander getrennt, sowohl durch die Ansprache im User Space als auch durch die Abstraktion einer Warteschlange für Anfragen, welche Ein- und Ausgaben strikt trennt.

Queue-based Load Leveling

Durch Warteschlangen in den diversen Subsystemen gelingt es dem Linux Kernel sowohl mit hohen relativen als auch absoluten Lasten umzugehen. Sowohl im Zugriff auf Geräte, als auch im Zugriff auf CPU Leistung werden Anfragen nicht direkt ausgeführt, sondern in einer Warteschlange angehängen.

Caching

Eine weitere Strategie im Umgang mit hohen Lasten ist die Implementierung diverser Cache Mechanismen im Kernel. Lese- und Schreiboperationen werden mit dem Arbeitsspeicher als Cache ausgeführt, und nur sofern nötig tatsächlich persistiert.

Desweiteren geschehen diverse Operationen, welche das Dateisystem und seine Metadaten betreffen, ebenso erst im Arbeitsspeicher.

Überwachung

Der Linux Kernel stellt diverse Programme bereit, um Laufzeitverhalten des Kernels zu überwachen.

Watchdogs können harte und weiche Deadlocks erkennen. Sanitizer können diverse Programmierfehler erkennen (z.B. Kollisionen oder use-after-free), werden aufgrund der Leistungseinbuße meist nur für Entwicklungszwecke benutzt.

Erkannte Fehler resultieren meistens in Warnungen "Oops", oder in kontrollierten Systemabbruch "Panic", je nach Konfiguration des Kernels und Umgebung des erzeugten Fehlers.

Experimentelle Ergebnisse

Im Rahmen der praktischen Experimente ist es uns gelungen einen existierenden Gerätetreiber für ein "Block Ram Device" so zu modifizieren, um diverse Fehlerszenarien provozieren zu können und die Reaktion des Kernels zu beobachten.

Wir demonstrierten die folgenden Szenarien:

- 1 Gerätetreiber hängt endlos: Andere Geräte unter dem selben Treiber bleiben ansprechbar
- 2 Gerätetreiber gibt einen Fehler zurück: andere Treiber bleiben unbeeinflusst
- 3 Ein NULL Zeiger wird dereferenziert: In einer sub-routine wird der Fehler toleriert, im Kontext der Abwicklung ein IO-Anfrage, deutet der Kernel diesen Fehler als fatal und verursacht eine Panik.